Complementary Threat Knowledge in Federated Android Malware Detection

A Controlled-Exposure Study

Salaheddine Naslouby[1,*], **Mohamed Lachgar**[2], **Hamid Hrimech**[1]

[1] Hassan First University
National School of Applied Sciences (ENSA), LAMSAD Laboratory
Berrechid, Morocco

[2] Cadi Ayyad University
Higher Normal School (ENS), L2IS Laboratory
Marrakech, Morocco

* Corresponding author: Salaheddine Naslouby
Email: naslouby.salahe@gmail.com; ORCID: 0009-0002-7295-7816

Mohamed Lachgar: m.lachgar@uca.ac.ma
Hamid Hrimech: hamid.hrimech@uhp.ac.ma

Abstract

Training collaboratively allows us to detect malware families that we didn't see during local training. However when looking at federated vs local training it also factors in the benefits of pooling data. Complementary threat knowledge (CTK) was defined as the amount of recall when your peers contain your hidden families compared to training without any clients having those families with the same budget. When testing with 4 android market/era clients from LAMDA and AndroZoo. With leakage aware partitioning, per client calibration and 10 confirmation seeds. FedAvg gave us a CTK of 11.96

Introduction

There could be thousands of samples for an android malware detector and none from a family you care about. Apps will continue to come from various sources with families, benign populations and labels your detector has not seen. So at some point your detector will come into contact with a family it's not trained on. Maybe other peers have this family but we don't know if federating with these people will help your detector learn that family better then just having those samples alone.

Federated learning can learn from multiple clients by averaging models learned from the data each client has. When looking at a federated model compared to a local model there are many variables at play. How many clients are involved, how much data was learned from and what families were available to each client. You can see a decrease in the performance of a federated model while gaining the knowledge of a family not found on that client or the other way around.

This leads us to define complementary threat knowledge (CTK). Given a client that does not have a family ctk is the difference in recall when training using peers that have a family versus no clients having that family. The difference between the model that doesn't have access to that family and the local detector is pooling. These two can explain why a detector learns from collaborators better then it's local counterpart or see pooling effects lower than zero that are made up for by the knowledge of complementary threats.

Using LAMDA features and androzoo information we created four clients of market and era. Each had families withheld and we looked at detection as a binary problem with benign on each client. Our splitting scheme accounted for groups and kept packages and the same first level feature vectors out of the fit, calibration and test set. We kept amount of data the same for all clients that had exposure to a family.

In this chapter we will show how federated learning of Android Malware can help find families not found on that client by creating equal amounts of training data for clients that have that family. We will look at how much ctk can exist when federating and use controls and doses to give confidence that our claim is accurate. Finally we will dive into families and clients to see where we lost knowledge of other malware.

Background and Related Work

Android Malware Detection and Family Generalization

These static features are commonly extracted from manifests and code and have been used to help identify if an android application is malicious or not and determine why an application may be considered malicious (Arp et al., 2014). These classifiers would know that an application is malicious but not know which family it may belong to. So we could see these classifiers learning about families that they have not seen on their local machines prior to working with others. We prevent this from happening in CTK-Android by withholding families.

By changing the representation you can allow more structure to be learned between families. By abstracting api-calls to packages/families and learning markov chains on those features (Mariconti et al., 2017) MaMaDroid was able to identify structure in families. Some families may not have any learned structure when using one representation but if you extend the representation they may show some structure. Which means that CTK can be compared over multiple representations not just by how many features you have added.

Family labels can also be ambiguous. AVClass2 is an approach that assigns families using pre-defined tags and rules from anti-virus labels (Sebastián & Caballero, 2020). Many datasets use these methods to determine the family an app may belong to. Some of these families could contain similar traits such as containing an advertisement package. CTK-Android uses the families given to it as true.

Federated Learning and Complementary Exposure

FedAvg will combine each round's local models together and FedProx uses a proximal term to restrict client deviation from the global model (Li et al., 2020). We will be using both to compare against because when models are combined there could be a negative affect to the clients that do not have all android families.

When training to classify malware there has been research that looked at differences in the distribution and model aggregation. Ciaramella et al. (2026) look at model aggregation with federated learning versus centralized learning. Using application code as images and CNNs they

test various scenarios of differing distribution. Gálvez et al. (2021) investigate privacy and Patel et al. (2026) investigate client level drift. M2FD used federated learning on mobile malware and can monitor for drift on the KronoDroid dataset (Augello et al., 2025). Droidware incorporates federated learning into malware detection on static features and monitors model authenticity and outlier clients (Prasad & Kumar, 2026). All these papers show how you can use federated learning on malware but don't investigate if a client benefits from other clients having more knowledge of android families.

There are other works that investigate similar questions to this research. Ongun et al. (2025) investigate federated learning to detect HTTP Threats. They create CELEST that looks at the effectiveness of federated learning when the threat family is available in other clients. Along with investigating active learning and preventing malicious clients. Similar to this paper is CyberForce which varies the number of clients without a certain malware in a federation of IoT devices. Including one experiment where no clients have the targeted malware. Then testing multiple aggregations (Feng et al., 2025). Shen et al. (2024) look at Intrusion Detection when each client doesn't know about one class. Popoola et al. (2022) look at unseen bots. And Shao et al. (2026) look at how your model can learn to a point of convergence but be skewed if positive cases are missing. CTK-Android mimics some of these designs to Android by adding equal amounts of replaceable samples to each client. It then measures the recall when many families are not known to each client while providing standard deviations of each seed. We also added a negative case that had all the incorrect family information for comparison.

Dataset Quality and Evaluation Validity

The AndroZoo dataset provides us with millions of apps that can help us learn more about malware then a few hundred. We only use AndroZoo to help build out our domains. When using the market to identify our domains we lose many apps that only have one market. Creating a federation that uses these domains does not represent how app stores may act.

There have been recent works that collect more Android datasets over a larger span of time (Haque et al., 2026) and have tried to bring together many methods of collecting data on the same

apps at different times (Kamol et al., 2026). We will use these as inspiration for our data set and how to learn from the data.

When evaluating your classifier you need to be careful on how you learn over time and space (Pendlebury et al., 2019). Sampling bias and information leaking are two common errors when building machine learning models for malware (Arp et al., 2022). CTK-Android uses a partition that avoids the same apps leaking information. But does not prevent apps from leaking information on a temporal level which has been studied in Android FL by Patel et al. (2026).

Research Problem and Questions

A seed specifies a group-aware partition and selected families' target assignments. For learner m, $R(m, P)$ is a specified recall metric when targets lack their assigned families and peers retain them; $R(m, A)$ is the same metric when all selected hidden families are absent from every client's fit data. $R(L, P)$ is the recall of the local learner on the same target-family task, although peer data do not enter its training. All quantities share the seed, nominal operating point, and evaluation population:

$$\mathrm{CTK}(m) = R(m, P) - R(m, A)$$

$$\mathrm{Pooling}(m) = R(m, A) - R(L, P)$$

$$\mathrm{Total}(m) = R(m, P) - R(L, P) = \mathrm{CTK}(m) + \mathrm{Pooling}(m)$$

All quantities are calculated over seeds and then averaged. As you can see CTK and pooling can both be less than 0. The learners, train schedule and amount of data given to each client is held constant between federated arms. Withholding different families will give different examples to meet this client budget. So CTK captures all these effects including seeing family and replacement. Since multiple families are being withheld CTK for a family can also be effected by what other families were exposed to. Pooling also contains the effect of different train schedules between federated and local.

There are 4 research questions addressed in this chapter. Can a client gain recall on an unseen family from their peers exposure to a hidden family? RQ1 will test if we see this trend with no pooling. We will look at client macro recall of unseen families over all clients in a federation for FedAvg and aim for a nominal FPR of 5%. How does this vary by family and which clients see a boost in performance greater than their pooling loss? What learning strategy will yield more unseen family recall at the cost of nonhidden-malware recall, FPR and clients with low recall? Does our results still hold with different families, family support, partitions, learner class, aggregator and representations. Full exposure where all families are seen will act as an upper bound for our learner/budget. More exposure may be able to outperform full exposure if trained for longer or samples are collected better.

Controlled-Exposure Study Design

Data Sources and Constructed Clients

This is from the LAMDA var_thresh_0.01 release it consists of 1,008,381 applications with 925 static features. All features are positive so they were just binarized no further feature extraction/vocabulary was created. As per the source any application that has 4+ antiviruses flagging it is considered malware and 0 is considered not malware anything in between was discarded. The source contains data from 2013-2025 there are no apps from 2015 (Haque et al., 2026). We create our own train test splits.

Each sha256 from the dataset could be matched to one record in the androzoo metadata table with a matching virus total count. There where only apps that are tied to one market used. Anzhi and appchina both make up one client and google play was separated into play-early which contained apps before june 2019 and play-late which are all apps after june 2019 the rest of the apps were not used. This gave us a total of 832,165 apps 257,564 are malware and 574,601 are not.

The class ratios vary greatly between clients (Table 1). For instance we see that Anzhi and AppChina have high amounts of malware classes whereas our two Google Play clients are comprised mainly of benign classes. We did not balance these ratios so a larger increase in recall could effect

Table 1
Retained Applications in the Four Constructed Clients

Client	Applications	Malware	Benign	Malware (%)
anzhi	104,287	90,801	13,486	87.07
appchina	88,282	73,578	14,704	83.34
play-early	319,122	75,985	243,137	23.81
play-late	320,474	17,200	303,274	5.37

Note. Corpus counts before grouped splitting and fit-budget selection. Client labels denote constructed market/era domains; percentages describe this selected corpus.

each client differently. These ratios only apply to our chosen corpus they do not reflect ratios on the market place.

Every corpus is greater than the number of samples we can train on. In our default setup we will be able to train on at most 6,000 samples per arm (1500 samples for our lower support sensitive setting). Our calibration and testing set remain consist of all of there sample partition groupings. So we will always have the same amount of data to train on for each arm and could have more peers available.

Identity Grouping and Partition Construction

We group together any applications that had the same package name or had a matching primary feature vector. All applications that are transitively connected by these relationships form an identity group and will be assigned to a single role for the duration of our research. There will never be a package or primary feature vector in more than one of the fit/calibration/test sets but all clients will appear in each set. We assign identity groups to fit/calibration/test in order of seed, salt and number of attempts until we reach the target proportions of 60%/20%/20%. Since each identity group must be wholly contained in a single set our final proportions may vary slightly. We create 16 possible partitions per seed and salt then select the partition that maximizes number of clients with enough support to be a fit target and natural scarcity family target. In case of a tie we take the first partition. Our method only considers support not classification performance. The partitions we chose across the confirmatory seeds all have between 57-60 eligible families as clients compared to

a median of 52.5-56 across all candidate partitions, this provides roughly 9% more support for our models. Results should be interpreted under the knowledge that we selected for this.

The splits are not based on time. Both google play markets have their own fit/calibration/test split meaning that just because an application is older does not mean it was fit on prior to being tested. We expect to generalize to new identities on each market/time period. Grouping solely on package name and removing the feature vector relations provides an additional sanity check. It does not preclude leakage of other information.

Family Selection and Target Assignment

We do not include empty, unknown, benign or singletons as target families but those applications will be used in our binary task. Any family that has 600 or more applications in our filtered data set and appears in 2 or more clients will be a target candidate. We take the top 8 families with the highest amount of support for each set and assign them deterministically by taking the top ranked family from the set then the next lowest from the secondary. The target family list consists of the following 8 families with the first 7 meeting our criteria for the controlled study. frmy will also be a target on the replication set.

We consider a target/family combination valid if there are 20 fit examples of the target family to be excluded from that target client. There need to be 150 examples of the target family from the rest of the client base as well as 50 test examples on the federal level. Additionally after excluding the family we need 500 fit examples left on the target client. This total is the sum of all example fit applications for the families assigned to that client. We will pick clients in seed order until we find a valid target for that family. Thus a family can be on different targets depending on seed.

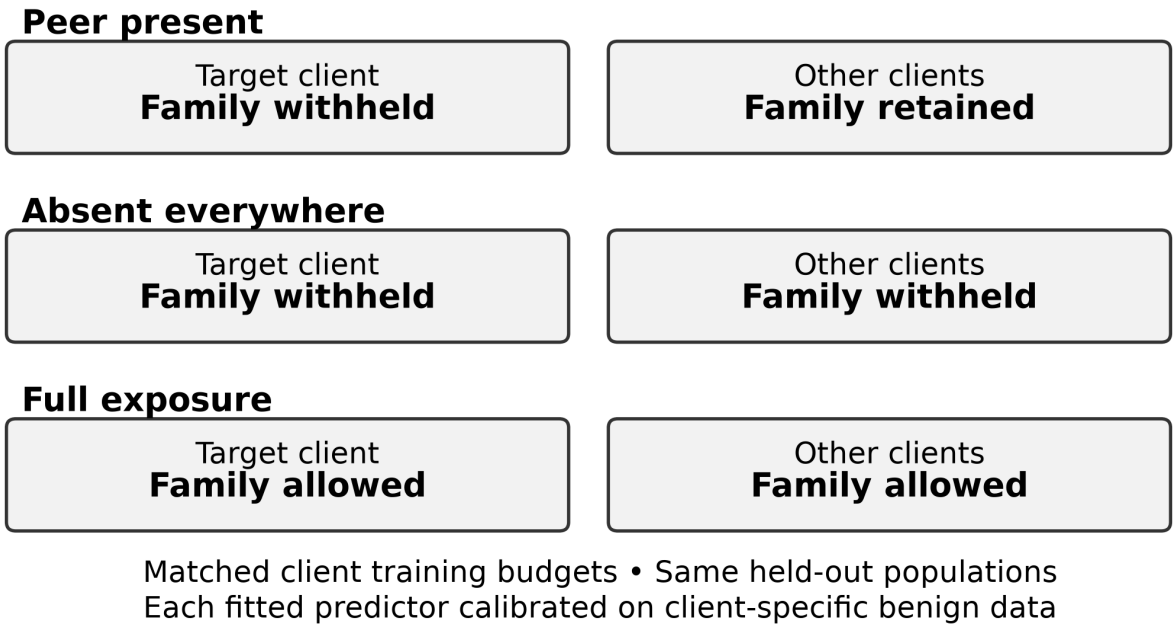
If reporting on the targets own domain we require 15 test examples of the target family on that clients domain. A family may be used in the federal report but not on the targets own domain. As a result we will have a varying number of clients/seeds used to report our metrics.

Exposure Conditions and Comparison Designs

Figure 1 depicts the various conditions we will compare. With Peer-Present the selected family is only excluded from it's target. For Absent Everywhere we will exclude all selected families

from every client. Full Exposure leaves no families excluded from any client. We then use seed order to fill the clients budgets with the remaining data resulting in equal sample sizes for each condition.

Figure 1
Controlled Family Exposure Across Target and Peer Clients



Note. The schematic illustrates one target-family pair. Each run can assign several families to different targets. In the peer-present condition, a selected family is withheld only from its assigned target, which may retain other selected families. In the absent-everywhere condition, all selected families are withheld from every client. Replacement examples preserve the 6,000-sample training budget per client. Full exposure removes the deliberate exclusions but retains the budget.

The sample size per client stays the same if we swap out the holdouts for other examples. However the amount of malware/benign each client has will vary. We also recalibrate using their own benign examples for each arm. So CTK represents the exposure+swap+recalibration effects of our selected families together and does not represent a single family.

We have our naturally low example scenario which tests if our target family has <5% of their family's federated example fit. We require our peers to have at least 150 fit examples and the target to have 15 test examples. Because the target family is not removed there can be some local train

examples that remain. We also choose different families and definition of scarce compared to our controlled scenario. Thus it's more of a sensitivity analysis than our equivalent scenario.

Aligned Representation Extension

By taking our LAMDA applications that remained after filtering and finding them in McNdroid through application hash we have 554,136 apps. With 170,256 malware and 383,880 benign apps. We test 4 different representations on this common pool of applications. 925 lamma static features, 2,390 McNdroid static features, 2,793 call graph columns and 17,483 report json columns. Our graph columns are those created by McNdroid. There is no newly created graph representation. The report features are signed log. The call graph and report columns are kept if they are above 1% in the train set. All columns are normalized using the train statistics. Each model is fitted with their own train rows these are client rows if local or the arms rows for Central and FedAvg.

Our representations will keep the same label set, client set and families that we have in our common pool. We focus on hiddad, gappusin, revmob, leadbolt, airpush, dowgin and adwo as they are families we looked at before. Which makes these an inside comparison.

Learning and Evaluation Protocol

Learners and Frozen Training

Table 2 lists the frozen settings. The base model is an MLP trained with binary cross-entropy on logits. Local models train for 10 epochs on their client's budgeted fit sample, and the central model trains for 20 epochs on all four budgeted samples. FedAvg runs 20 rounds in which all 4 clients start from the current global parameters, train 2 local epochs with a newly created Adam optimizer, and are averaged with weights proportional to fit examples (equal here, because every client fits 6,000). FedProx adds a proximal term; fine-tuned FedAvg trains the final FedAvg model for 2 more epochs on the target's fit data; and the blend averages local and FedAvg predictions with equal weight. Every predictor is calibrated on its client's benign calibration scores. Local epochs, fine-tuning length, and proximal strength were chosen on a development set rather than on test results, and they do not guarantee that each learner reaches its optimum.

Table 2
Conditions of the Primary Controlled Study

Component	Setting
Source and representation	LAMDA var_thresh_0.01; 925 binary features
Partition	Package/feature connected components; 60/20/20 fit/calibration/test; best support among 16 attempts
Training volume	6,000 fit rows per client; 1,500 in lower-support sensitivity
Primary eligibility	Target fit ≥20; peer fit ≥150; federation test ≥50; remaining target fit ≥500
MLP and optimization	256/64 hidden units; rectified linear activation; dropout 0.1; Adam; learning rate 0.001; batch 256
Training schedule	Local 10 epochs; central 20; federated 20 rounds × 2 local epochs
Adaptation	FedProx $\mu = 0.1$; fine-tuning 2 epochs; equal probability blend
Operating points	Nominal FPR 5% primary; 1% and 10% sensitivity
Primary inference	Seeds 100–109; paired effects; 9,999 BCa resamples; exact Wilcoxon; Holm across three contrasts

Note. Support requirements refer to pre-budget pools. Family-macro, own-domain, and worst-client summaries use distinct denominators. Hyperparameters and statistical procedures were frozen before confirmatory evaluation. The stored resolved configurations of all confirmatory runs match the frozen run-level protocol.

There are also two other model classes used to see if the difference relies on the MLP. One being a linear classifier trained locally, centrally and with FedAvg. And the other a histogram gradient boosted tree learner trained with up to 100 estimators and depth of 6 trained locally and centrally.

Calibration and Metrics

The threshold for a given client and arm is calculated by the $(1 - \alpha)$ empirical quantile of their benign calibration scores. Using the upper order statistic any application above the threshold is alerted. Our primary cut-off is $\alpha = .05$. And we explore sensitivities of .01 and .10. If there is not enough support in the client we will have insufficient calibration if $n_{\text{benign}} \times \alpha < 10$. There may be ties in scores and test/calibration population mismatch will lead to different FPRs that hover around our nominal value. So on our primary cut off we require the test FPR to be within 2 percentage points of our nominal value on the run level for client macro. Note that this does not restrict an individual clients FPR.

We report on our clients own domain recall for applications that are unseen families. Averaged over all targets that meet our criteria for own domain support. We also compute the recall on unseen families over all domains for our targets clients detector and average this. Federation-wide recall gives us more support but these applications may not be in our marketspace or time period. Family macro is reported to average over families that our targets qualify for and give equal weight to all families. Micro pooling our clients will result in more weight given to families with more support. The primary estimand of interest is client-macro federation wide. Though family macro is reported on our dose and family specific results. You may see different numbers depending on how these metrics are averaged.

Our recall on known families is measuring recall of malware that is not hidden for each client. Which may contain unseen families without a label. We measure achieved FPR on the client mean of their benign test applications. And our worst clients FPR is the maximum achieved FPR of any of our clients. For our worst clients we calculate their federation-wide recall on unseen families. Taking the minimum target client that qualifies for this metric. As you can see our worst client may differ by arm and thus these numbers are not comparing one clients increase in recall.

Our AUROC and AP will be averaged over our clients. Taking the mean of each clients result on all applications in their own domain test set. Average Precision calculates the sum of precisions instead of using the trapezoid rule to interpolate between precision@recall values. It will depend on class proportion. Note that these numbers are not computed over only our unseen families nor our operating point.

When we state difference in recall we mean the percentage point difference. All recall and false positives rates will be presented in percentages. Nominal False positives will refer to the rate we aim for when calibrating on the clients benign applications. Achieved FPR will be the rate on there benign test applications. All intervals will be reported as 95% confidence intervals for the mean using BCa.

Statistics

Our seeds for our confirmatory questions are 100-109. Each seed will have its partitions and targets fixed for each arm. So when looking at a difference we will report on our mean, median, amount of positive seeds and standardized effect size. Calculating our 95% CI using BCa and bootstrapping the seed differences 9,999 times. Since there are no zeros or ties in our absolute difference between arms for our primary questions we are able to perform a 2-sided Wilcoxon Signed Rank Test. To determine if our distribution is centered around 0 or not. It does not tell you if your mean difference is 0. We correct for multiple comparisons using Holms (Holm, 1979) on the three FedAvg questions of federation wide CTK, pooling and total at $\alpha = .05$. All other learners and estimands are not corrected for as well as our family questions and diagnostics.

We pre define our cut offs for practical purposes when evaluating our models. 3 percentage points of CTK would be considered beneficial to our product. And our false positives rate may decrease by 2 points on recall of malware that are not hidden families. Using bootstrapped CIs and signed rank tests answer different questions so we decided not to modify our methods based on if they agreed or not.

A - Original Confirmatory Questions.

B - Sensitivity Questions Predefined.

C - Post confirmatory analysis of information already saved.

D - New questions asked on frozen code that have been prospectively extended and diagnostic questions asked after the fact.

Extensions were done with new seeds. Seeds 200-209 used to calculate the p-value for the permutation test. Seeds 300-309 were used for our additional families extension. Seeds 310-319 will calculate our exact dose response question. Seeds 320-329 will calculate the placebo and aggregation effects. Seeds 330-339 will look at representations of clients. They should not be interpreted along side seeds 100-109 as confirmatory evidence.

All code used to reproduce this paper and fingerprint of the data is available at https://github.com/naslouby-salahe/ctk-android. This paper is reporting on run rev. f6302f7906824945731dadbbeb44357f3d025aaf.

This paper reports results from our last run that was completely deterministic with drop out fixed to a seed and kernels deterministic. Previous runs were also set up deterministically but could not reproduce results exactly bit for bit. Which produced FedAvg CTK numbers ~11.7 points. These will not be discussed further and our current run will not pool these previous results. Source Data must be obtained from its provider.

Results

Controlled Exposure

Having access to malware from your peers increases your unseen family recall as seen in table 3. On average our fedavg model increased its federation-wide recall of unseen families when they were absent in every client from 52.41% to 64.36% when they were available through its peers. When our clients had local knowledge of these families it improved its unseen family recall to 50.90%. Having all families represented resulted in 67.64% unseen family recall. False positives stayed around the same level in each condition. Average FPR of 5.04%, 5.09%, 5.04% respectively for our absent in all clients, present in peers, and present in all clients. There can be some clients that exceed our nominal value as you can see in Table 3 our mean worst client FPR when our peers knew our hidden families was 6.60%. And our highest false positive was 9.89%.

Table 3
Primary Paired FedAvg Decomposition at Nominal 5% FPR

Contrast	Mean (pp)	95% BCa CI (pp)	Holm *p*	Positive seeds
CTK	+11.96	[+9.15, +15.21]	0.00586	10/10
Pooling	+1.51	[−4.15, +6.10]	0.32227	7/10
Total	+13.46	[+8.82, +17.04]	0.00586	10/10

Note. Ten confirmatory seeds, 100–109. CTK = peer present − absent everywhere; pooling = absent everywhere − local; total = peer present − local. Realized FPR: local 4.99%, absent 5.04%, peer 5.09%, full exposure 5.04%. Intervals summarize seed-paired mean differences; *p* values are exact two-sided signed-rank results adjusted across these three contrasts. pp = percentage points.

All seeds had a positive value for CTK and total gain. Pooling interval showed a mix of small negatives and positives. An average of 2,073 out of 24000 samples varied between the arms of peer present everywhere and none.

Our results for own domain unseen show a positive effect for fedavg ctk at 10.85 (6.97–15.25), pooling was -7.60 (-11.81, -4.03) with a total gain of 3.26 (1.26, 5.63) compared to local learning alone. In other words our estimates for own-domain CTK make up for most of what was lost with pooling. Note these values are only estimates for our secondary endpoints and have not been adjusted by Holm correction.

Similar patterns can be seen when pooling differently. If we take our macro average over families of malware we get a value for CTK of 12.44 (10.09–15.55) with a total gain of 10.57. The worst client also had a positive value of 18.49 (13.88–26.29) and a total of 14.88. Our estimate when weighting all hidden test applications equally was a smaller number at 6.58 (4.64–8.65) and again positive in all ten seeds. Pooling was estimated at 4.00 and total at 10.58. This lower number is due to dowgin accounting for roughly 47% of our hidden family test apps. Dowgin does not produce many positive values for CTK so if we exclude this we see our micro-pooled estimate increase to 12.24 points. At different FPR estimates we also saw similar patterns with our federation wide recall being higher at 7.85 points for 1% nominal fpr and 13.78 points at 10%.

Learning-Strategy Comparisons

We compare the various learners under peer-present conditions in Table 4. All learners had positive CTK in all ten seeds. Central training was 13.10 points higher then FedAvg. FedProx had an increase of 13.20 points while the fine tuned version of FedAvg increased by 8.20 and blending increased by 7.60 points. We cannot differentiate between FedProx and FedAvg since their difference for unseen recall was only 0.19 points (-1.12, 2.94). Descriptively central training has a larger federation-wide average recall but FedAvg had a larger average for worst client.

For Nonhidden-malware Recall there was a change in ordering. FedAvg had a drop of 3.61 (−4.50, −2.85) when compared to local learning and the fine tuned fedavg dropped 3.74 (−5.31, −2.82). Both drops were above the threshold of 2 points we decided on. FedProx decreased 0.93 and

Table 4

Peer-Present Learner Performance at Nominal 5% FPR

Learner	Federation unseen	Own-domain unseen	Worst client	Nonhidden	FPR
Local	50.90	50.67	28.92	73.06	4.99
Central	66.07	56.27	41.42	72.92	5.10
FedAvg	64.36	53.93	43.79	69.46	5.09
FedProx	64.55	55.98	37.90	72.13	5.09
Fine-tuned FedAvg	62.16	51.64	40.35	69.33	5.03
Blend	63.66	57.21	37.63	74.77	5.12

Note. All entries are percentages averaged over ten confirmatory seeds. Worst client is the within-seed minimum federation-wide unseen recall. Other recall and FPR columns follow the macro definitions in the protocol. Arm rankings are descriptive; table levels are not confidence intervals or adjusted pairwise superiority tests.

our centralized training dropped 0.14 while our combination increased by 1.70. As these are means being compared to each other with a certain threshold we can not make claims about equivalence. We see that FedProx and our combination kept a higher mean for nonhidden malware but a lower min client.

As expected there was a slight decrease when comparing our results to Hidden families. Mean AUROC for own domain data changed by -0.01166 when using FedAvg over local and Average Precision by -0.02011. When evaluating your detector you need to look at both.

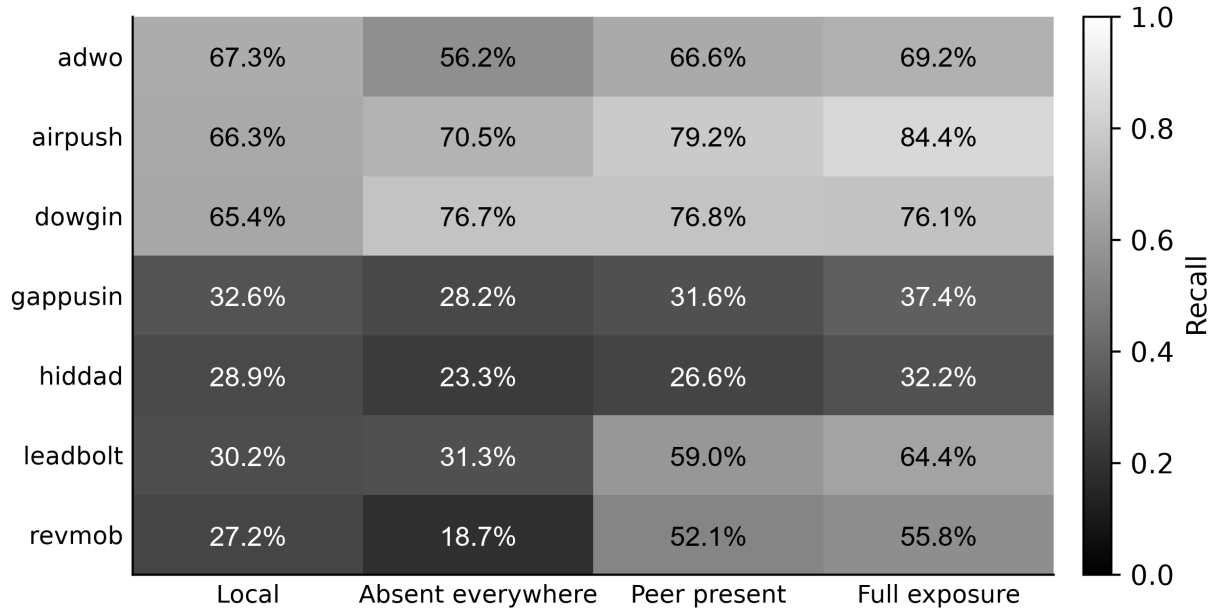
Malware-Family and Client Heterogeneity

Figure 2 plots recall against primary family of malware. There is an increase for Revmob and leadbolt with a CTK of 33.38 and 27.75 respectively. Revmob still does not reach 60% recall when exposed to all the families. So it can take a high amount of points and still not be enough.

The effect varied between families. For dowgin we saw improvements from pooling alone. It had high recalls compared to local even when it was not present anywhere. It's CTK was 0.14 points. adwo demonstrated improvement from negative transfer. With a CTK of 10.37 points nearly cancelling out it's decrease in recall for the not present anywhere arm. Though still lower then local when it's present with other peers. gappusin and hiddad improved slightly from being represented statically. Their CTK was 3.46 points and 3.25 points respectively. Both were lower then local recall

Figure 2

Family-Specific Recall Across Exposure Conditions



Note. Cells show mean recall percentages to one decimal place at nominal 5% FPR across seeds 100–109. Full exposure refers to FedAvg and is a finite reference. The grayscale bar expresses the same recall values as proportions.

when present with other peers at 31.63% and 26.57%. With full exposure to federated average they were able to get a recall of 37.41% and 32.18%. These two families could benefit from better representation. airpush fell in between these responses with a CTK of 8.71 points and a recall of 79.24% when present with other peers.

For our replication data we found CTK to be -8.24 points for dnotua and -2.06 for zdtad. And 13.63 for inmobi and 12.04 for domob. utchi and zdtad had 6 seeds that met this criteria. Overall the aggregate showed improvement from exposure. Some families will see an increase others will not.

Per Client(Class C) we found federation wide CTK using Federated Average to be 8.35 points at Anzhi. 7.58 points at AppChina. 9.18 points at play-early. And 32.03 points at play-late. With seed counts of 8,10,10 and 6 respectively. Play-late has a higher number due to it having a smaller amount of seeds and around 107 hidden test apps that are in it's own domain. Overall Anzhi only increased it's recall by 1.34 points and dropped it's recall on nonhidden-malware by 6.93 points. Our overall domain results for these clients were -2.80 points at Anzhi. -0.40 points at AppChina.

6.48 points at play-early. And 13.61 points at play-late. All with confidence intervals that cross zero. Showing that not all clients will increase there own domain recall.

Peer Exposure and Dose Response

When requesting peers from the cap we did not reach the expected dose. When our caps were set to 100 and 1,000 the average dose using Federated Average was 9.03 fitted peers and 79.39. Because we cap the number of peers before we sample them. We got 296.19 fitted peers with our all dose and achieved a CTK of 12.44 points for family macro. Only when using all peers did we exceed 100 fitted peers per family.

We also looked at our extended dataset to get a fixed dose response. The family macro federation wide CTK using Federated Average was 2.92 points. 4.90 points. 7.83 points. 12.48 points. And 14.49 points when using 10, 25, 50, 100 and 200 effective doses of peer information (Figure 3). These CTK's were all positive over our 10 seeds. Using a dose of 10 we have a CI that falls under our 3 point cutoff. With 25 it reaches above the cutoff.

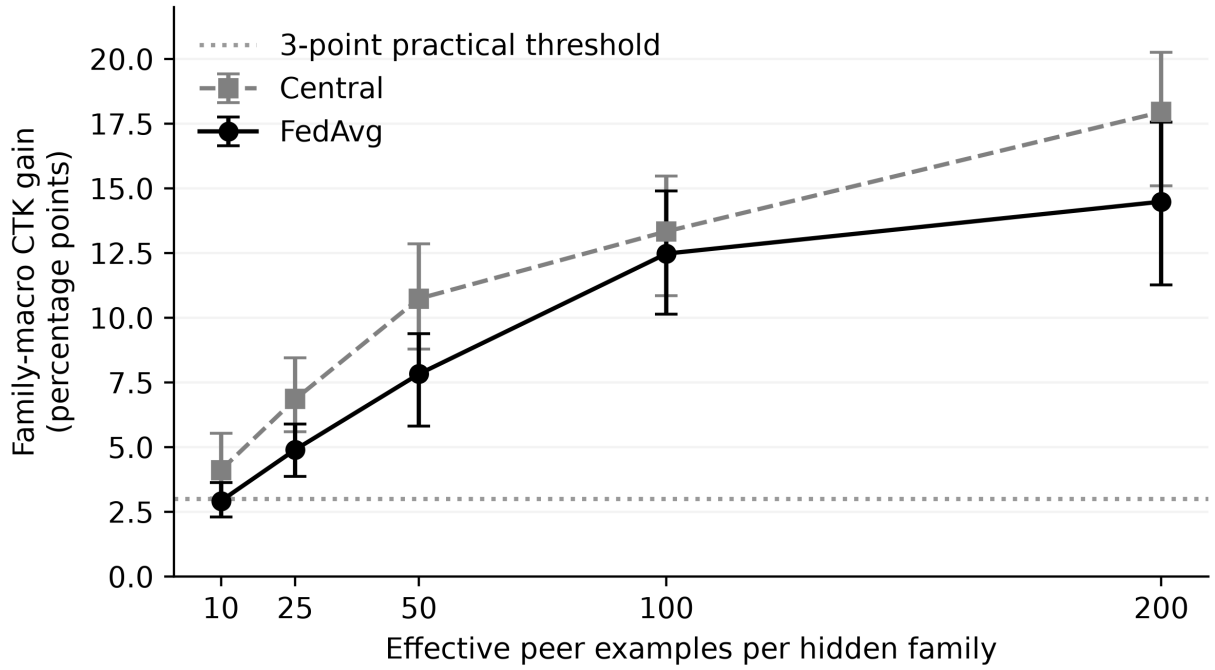
It never seems to plateau. When we looked at the pooled CTK it went up by 4.65 points from 50 to 100 (2.43-7.09) and 2.01 points from 100 to 200 (−0.06-4.06). The main group continued to increase by 4.23 points from 100 to 200 (2.53-6.55) but the replication group did not change by 0.03 points (−3.91-2.54). There does not appear to be a dose where families plateau or reach a minimal effective dose. Dowgin was around zero or less for all doses including the highest. The zero dose group is equivalent to none by design.

Negative Controls, Replication and Sensitivity

The family placebo shuffled the associations between the families and application while holding the malwaredomian association constant. We found that when using our original FedAvg the federation wide CTK was 0.30 points (−0.36-1.02). This is considered to be zero under our criteria of being within 3 points. Our own domain CTK had a 3.04 upper bound that is undetermined. Using the extension both federation and own domain CTKs were below our threshold at -0.21 and -0.07 respectively.

Figure 3

Exact Peer Dose and Complementary Gain



Note. Prospective extension, seeds 310–319. Family-macro federation-wide effects pooled within seed across primary and replication sets at nominal 5% FPR; 95% BCa intervals. The dotted line marks a practical threshold, not a universal dose requirement.

We report replication, sensitivity and negative control results in Table 5. As previously mentioned when we took away support from the wrong family we received a score of -0.54. All ten of our extension runs showed an increase when subtracting placebo from CTK. Showing that by giving our algorithm the right family it can improve.

Disjoint replication family set was positive for 8/10 seeds with a mean of 5.08 points of CTK. The interval did not stay fully above the 3 point mark. We also got positive means with linear fedavg, central tree's, lower train support, lower and higher family support, group by package only and three other salt choices for partitions. Positive mean ctk for natural scarcity as well with differences in local seen and target families and pool size. Again these check our claim for the same corpus and family creation process.

When using robust aggregation our pooled ctk is 9.50 points when using the mean. When we trim off the largest and smallest score from each of the 4 clients and take the mean of those scores

Table 5
Selected Replication, Sensitivity, and Control Effects

Comparison	Class	Mean (pp)	95% BCa CI (pp)
Disjoint family set	A	+5.08	[+2.28, +7.10]
Natural scarcity	B	+11.19	[+9.46, +12.59]
Linear FedAvg	B	+9.37	[+7.54, +10.97]
Centralized trees	B	+10.18	[+7.59, +13.70]
Lower training support	B	+10.99	[+8.71, +13.52]
Lower family support	B	+14.01	[+11.46, +16.22]
Higher family support	B	+11.59	[+9.17, +13.75]
Package-only grouping	B	+11.32	[+8.60, +13.29]
Wrong-family placebo	D	−0.54	[−1.36, +0.20]
CTK minus placebo	D	+10.04	[+8.35, +12.28]

Note. A = original confirmatory; B = prespecified sensitivity; D = prospective extension. First eight rows are CTK effects on the client-macro federation-wide metric, seeds 100–109. Last two are family-macro effects pooled within seeds across family sets, seeds 320–329. These rows are not one multiple-testing family and are not pooled into the primary test. A positive mean above three points need not have an interval entirely above that practical threshold.

we get 9.16. Using a coordinate wise median taking the lowest median score yielded 13.76 points. All of these methods are assuming our clients are being honest and do not look into defenses against malicious clients.

Representations and Extensions to More Families

By looking at representations we can tease out if an increase in full exposure recall and CTK are happening simultaneously or not. With the priority family macro the McNdroid static features give us an increase in central full exposure recall of 7.57 (2.76-12.85) points and an increase in fedavg CTK of 8.01 (3.56-10.54) when compared to aligned lamda. Call graph features increased ctk by 9.03 (5.53-12.35) points and full exposure recall by only 2.52 (-.86-7.77). So increasing CTK does not increase full exposure recall because CTK is relying on the recall of the missing families. Using the report json features we decreased our full exposure recall by 14.54 (-18.63 - -8.66) points and CTK by 6.12 (-9.91 - -3.87). Some families benefited more then others with the addition of representations. Static features gave us an increase in full exposure recall of 16.93 points for the family gappusin and 33.60 for adwo. For hiddad we estimated a -6.21 (-18.02 - 6.29) which was inconclusive.

All of the extensions looked at nominal calibrations not equalized fpr’s. We failed to satisfy a macro-fpr tolerance when looking at exact dose for our primary seed 317. The lamda static representation and McN static representation seeds 333 and the call graph seed 334 had one central client that was able to detect 15.32% of benign packages that had the peer present. As a diagnostic we equalized fpr on the test set by calibrating to the benign scores on test. Full exposure was increased by 7.28 points when using the static features and decreased by 14.27 with report json. The interval included 0 for the call graph full exposure. This is just a diagnostic not something you could use.

Our expanded set of 32 families had a mean family ctk of 6.51 points. Five families were negative including feiwo -5.89 and baiduprotect -4.41. The distance to the nearest family we have enough support for is correlated to CTK spearman .408 (family bootstrapped .088-.662 perm p-value .0214). When restricting to families that are eligible on all fresh seeds n=27 we see a weaker relationship .294 (-.076-.589 p-value .136). It is not directly related to new families but could tell us what types of partners will be helpful. There are 14 families that overlap with our primary or replication families. The remaining 18 families give us 3.47 points (2.45-4.45) of family macro ctk for each fresh seed. All fresh seeds were positive. Mean of 10.67 when overlapping families. Not a second replication but supports our findings.

Discussion and Real World Considerations

What We Can Say About Our Contrast

The amount of budget we give to both arms allows us to compare our found families with replaced samples and calibration that is unique to the arm. With the same training amount and number of samples from clients we cannot make that distinction with a local versus federated method. Complementary exposure made up the majority of our federation wide improvement. For your own domain pooling is hurting your recall and ctk helps recover from that. Clients will want to know if they are doing better or worse then their own model and why. Some families have properties that allow them to be found better with others some don’t. Adwo can recover from a drop in pooling, revmob and leadbolt can find families that are locally detectable. Dowgin helped without seeing any

of the target family and hiddad was difficult to find. Disjoint families and new families help as well. There could be different effects when changing corpora or representations.

Looking at the Different Metrics as a Package

Depending on your goal there are different populations you may want to evaluate. If you care about finding families on a federation wide level then recall over different source domains is important. How many apps is your client going to benefit from your learned model? Do you want to weight all your families the same or how many attacks can you prevent? Which learner gives you the most recall on your hidden families and how much are you losing on non-hidden malware. What is your fpr and recall on your worst client? We saw fedavg had the highest mean on the worst client recall and lost the most on non-hidden. Fedprox had a smaller drop in non-hidden recall but did not help on the mean unseen recall. Fine tuning and blending changed these trade offs even more. Decide what you can live without and evaluate on your populations.

Make sure to evaluate your FPR you may have clients that surpass the tolerance. Or that 5% of benign packages are being classified as malware. How much does that affect your triage process? Metrics such as AUROC and Avg Precision will miss families that you may care about. Recall over your hidden families may cause you to miss other metrics as well. Be clear what your population is and how your calibrating.

Seeing Others Help You Detect

You can measure your peers after exclusion and selecting the best budget for you. Knowing the nominal cap will lead you to think you are exposed to more families. Exact dose gives us the ability to see how many examples we will need when we encounter a new domain or use a new representation. As shown above representations can play a role in if we are able to find the families we are trying to detect. Higher CTK does not always lead to an increase in full exposure recall. Since we used the aligned representations these families overlap with our corpus and will only give us information on our family labeling.

We are not looking at privacy our clients are not being protected. Positive CTK when using trimmed mean and median show that in an unpoisoned environment we still are able to see benefits.

Correlation with centroid distance can lead us to think that families are helpful when we do not control for our exposure to those families. Testing on new domains that you may not share properties with will be necessary as well as unseen families and testing over time.

Threats to Validity

Construct and Internal Validity

We are comparing our findings to the family's they label. These may include different families that act similarly or combine multiple families into one. Our family selections skew towards adware and may not represent all of Android's malware. Locally unseen families are unseen by that clients fit set. We may not be able to distinguish between not having enough support for that family, not representing them well or our learned rule not capturing what we are looking for. When we leave out families we are leaving out more then one at a time. Our arms may affect one another due to the sample replacement we did. Our groups allow us to remove duplicates of a package name and top level features. But may not protect from similar package names or duplicate values in other features. Some providers share the same labeling scheme and had those dimensions removed from our partitioning. Best supported families of the 16 salt options was chosen. Creating 4 market era clients and using a modality intersection defines our population of interest. We are testing our models on these clients not predicting what will happen for a company.

Statistics and Data Validity

Using pairs of seeds we can see variations in our partitions and allocation. Training can vary as well on the same corpus. We have many repeated families and clients in our set of seeds. When we create smaller subsets of our clients we lose seed power. By dropping each of our seeds individually our ctk varied from 11.06 to 12.65 points. This gives us further evidence to support that we are above 3 points but does not replace our BCA interval. We only holm corrected our three planned comparisons. Worst client changes based on who our worst client is on each arm. Not telling us much about new companies. Lamda's database and labelers along with our markets may not be found in the wild today. Our labels for benign packages are based on no AV flagging that file. Other

labeling may give us different families. We were unable to test externally because McN is used in the dataset. Finger prints tell us what we used and not that it is representative.

External and Operational Validity

This work is done offline. We did not consider how long it takes to communicate with other clients or drain battery. How will this affect your analyst load or be run on device. How much privacy are you leaking to your peers. What happens when clients start trying to poison your model or evading your rules. You will want to collect these numbers if you want to use this method in production.

Conclusion

By exposing clients to other families that they can't find we increase fedavg's federation wide unseen family recall by 11.96 points (95% BCA CI 9.15-15.21) over no families being present anywhere. Our budget comparison showed us that we can attribute this increase to complementary exposure. There is still a question on how much pooling plays a factor in our results.

There are many considerations when looking at your families. Fedavg can improve your unseen family recall on your own domain by 3.26 points but lose recall on non-hidden malware by 3.61. Decrease your AUROC and Avg precision. We found that some families can out perform learning on your local data. Others can recover from the decrease in pooling and some remain difficult to detect. Our permutation tests and wrong family show that we need to be exposed to that family to find it. Exact dose allowed us to see if more exposure would lead to higher recall on those families. They all increased but no general pattern on how many exposures were needed to detect that family. Our models were tested on constructed clients and used calibrations on the client's data.

References

Allix, K., Bissyandé, T. F., Klein, J., & Le Traon, Y. (2016). AndroZoo: Collecting millions of Android apps for the research community. In *Proceedings of the 13th International Conference on Mining Software Repositories* (pp. 468–471). ACM. https://doi.org/10.1145/2901739.2903508

Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., & Rieck, K. (2022). Dos and don'ts of machine learning in computer security. In *31st USENIX Security Symposium* (pp. 3971–3988). USENIX Association. https://www.usenix.org/conference/usenixsecurity22/presentation/arp

Arp, D., Spreitzenbarth, M., Hübner, M., Gascon, H., & Rieck, K. (2014). Drebin: Effective and explainable detection of Android malware in your pocket. In *Proceedings of the Network and Distributed System Security Symposium*. Internet Society. https://doi.org/10.14722/ndss.2014.23247

Augello, A., De Paola, A., & Lo Re, G. (2025). M2FD: Mobile malware federated detection under concept drift. *Computers & Security, 152*, Article 104361. https://doi.org/10.1016/j.cose.2025.104361

Ciaramella, G., Martinelli, F., Peluso, C., Santone, A., & Mercaldo, F. (2026). A method for real-world privacy-preserving Android malware detection through federated machine learning. *Information and Software Technology, 189*, Article 107892. https://doi.org/10.1016/j.infsof.2025.107892

Feng, C., Huertas Celdrán, A., Sánchez Sánchez, P. M., Kreischer, J., von der Assen, J., Bovet, G., Martínez Pérez, G., & Stiller, B. (2025). CyberForce: A federated reinforcement learning framework for malware mitigation. *IEEE Transactions on Dependable and Secure Computing, 22*(4), 4398–4411. https://doi.org/10.1109/TDSC.2025.3547005

Gálvez, R., Moonsamy, V., & Diaz, C. (2021). Less is more: A privacy-respecting Android malware classifier using federated learning. *Proceedings on Privacy Enhancing Technologies, 2021*(4), 96–116. https://doi.org/10.2478/popets-2021-0062

Haque, M. A., Hossain, I., Kamol, M. M., Alam, M. J., Amalapuram, S. K., Talukder, S., & Rahman, M. S. (2026). LAMDA: A longitudinal Android malware benchmark for concept drift analysis. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=1FnCrZtBNQ

Holm, S. (1979). A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics, 6*(2), 65–70. https://www.jstor.org/stable/4615733

Kamol, M. M., Lopez, J., Nowmi, S. R., Rivas, E., Haque, M. A., Raff, E., Piplai, A., & Rahman, M. S. (2026). *McNdroid: A longitudinal multimodal benchmark for robust drift detection in Android malware* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2605.06894

Li, T., Sahu, A. K., Zaheer, M., Sanjabi, M., Talwalkar, A., & Smith, V. (2020). Federated optimization in heterogeneous networks. In I. Dhillon, D. Papailiopoulos, & V. Sze (Eds.), *Proceedings of Machine Learning and Systems* (Vol. 2, pp. 429–450). https://proceedings.mlsys.org/paper_files/paper/2020/hash/1f5fe83998a09396ebe6477d9475ba0c-Abstract.html

Mariconti, E., Onwuzurike, L., Andriotis, P., De Cristofaro, E., Ross, G., & Stringhini, G. (2017). MaMaDroid: Detecting Android malware by building Markov chains of behavioral models. In *Proceedings of the Network and Distributed System Security Symposium*. Internet Society. https://doi.org/10.14722/ndss.2017.23353

McMahan, B., Moore, E., Ramage, D., Hampson, S., & Agüera y Arcas, B. (2017). Communication-efficient learning of deep networks from decentralized data. In A. Singh & J. Zhu (Eds.), *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics* (Vol. 54, pp. 1273–1282). PMLR. https://proceedings.mlr.press/v54/mcmahan17a.html

Ongun, T., Boboila, S., Oprea, A., Eliassi-Rad, T., Hiser, J., & Davidson, J. (2025). CELEST: Federated learning for globally coordinated threat detection. *IEEE Transactions on Information Forensics and Security, 20*, 10434–10448. https://doi.org/10.1109/TIFS.2025.3614442

Patel, A., Tomar, D. S., Pateriya, R. K., & Haripriya, R. (2026). FL-MalDrift: A federated learning framework for malware detection under local concept drift. *Scientific Reports, 16*, Article 1821. https://doi.org/10.1038/s41598-025-31592-z

Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., & Cavallaro, L. (2019). TESSERACT: Eliminating experimental bias in malware classification across space and time. In *28th USENIX Security Symposium* (pp. 729–746). USENIX Association. https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury

Popoola, S. I., Ande, R., Adebisi, B., Gui, G., Hammoudeh, M., & Jogunola, O. (2022). Federated deep learning for zero-day botnet attack detection in IoT-edge devices. *IEEE Internet of Things Journal, 9*(5), 3930–3944. https://doi.org/10.1109/JIOT.2021.3100755

Prasad, A., & Kumar, S. (2026). Droidware: A security hardened federated framework for robust Android malware detection. *Engineering Applications of Artificial Intelligence, 181*, Article 115638. https://doi.org/10.1016/j.engappai.2026.115638

Sebastián, S., & Caballero, J. (2020). AVClass2: Massive malware tag extraction from AV labels. In *Annual Computer Security Applications Conference* (pp. 42–53). ACM. https://doi.org/10.1145/3427228.3427261

Shao, X., Otani, A., Yamakawa, K., Hirasuka, Y., & Wai Shiang, C. (2026). Objective mismatch in federated learning under missing class support. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*. Advance online publication. https://doi.org/10.1587/transfun.2026EAP1060

Shen, J., Yang, W., Chu, Z., Fan, J., Niyato, D., & Lam, K.-Y. (2024). Effective intrusion detection in heterogeneous Internet-of-Things networks via ensemble knowledge distillation-based federated learning. In *ICC 2024 – IEEE International Conference on Communications* (pp. 2034–2039). IEEE. https://doi.org/10.1109/ICC51166.2024.10622262

Key Terms and Definitions

Complementary Threat Knowledge: The detection benefit associated with peer exposure to selected malware families, measured here by comparing matched collaborative conditions with and without those families.

Federated Learning: Joint model training in which participants fit models on their respective data and combine updates through an aggregation process.

Android Malware Family: A supplied grouping of malicious Android applications treated as related for the purposes of exposure selection and evaluation.

Locally Unseen Family: A selected malware family absent from a target client's fit sample, regardless of whether its examples occur at peers or share features with other malware.

Controlled Exposure: An experimental manipulation of which clients' training samples contain selected family examples while preserving specified comparison conditions.

Residual Collaboration Effect (Pooling): The recall difference between collaboration without the selected hidden families anywhere and the corresponding local learner, including differences in training composition and optimization schedules.

Negative Transfer: A reduction in a specified target performance measure after a defined transfer or collaboration intervention, relative to its stated reference.

Realized False-Positive Rate: The proportion of benign test applications flagged as malware by the calibrated predictor, distinct from the nominal calibration target.

Effective Peer Dose: The number of selected hidden-family examples actually used to fit peer models after exclusions, availability limits, and training budgets are applied.

Family-Macro Recall: An average that gives each eligible evaluated target-family entry equal weight, rather than weighting it by its number of test applications.